

DSP in VLSI

Final Project

Eigen-value Decomposition by Iterative QR Operations

Student ID

M11407439 / B11107027

Name

MING-HONG, LIN / YING-CHENG, CHEN

Development Environment

HDL	SystemVerilog
Simulation Tools	Synopsys VCS and Verdi
Synthesis Tool	Synopsys DC Compiler
Process	TSMC 16nm (ADFP)
Verification	Python

Contents

1	Section 1: Wordlength and RMSE Analysis	2
1.1	Minimum Fractional Wordlength & CORDIC Stage Selection	2
1.2	Integer Bit-Width Analysis	3
1.3	Fixed-Point Bit-True Model	4
1.4	CORDIC Scaling Factor	4
1.5	CSD vs Constant Multiplier	5
1.6	Bit-True Model Verification	5
2	Section 2: Hardware Architecture and Control	6
2.1	CORDIC PE Architecture	6
2.2	Systolic Array-Based QRD Architecture	7
2.3	EVD Architecture	8
2.4	Detailed Transpose Buffer (Output Buffer)	10
3	Section 3: Hardware Simulation	11
3.1	Behavioral Simulation	11
3.2	Post-Synthesis Simulation	12
3.3	Fixed-Point vs. Hardware Consistency	13
4	Section 4: Area-Time Product	14
5	Section 5: Design Innovations	15
5.1	Angle-Free CORDIC via Rotational Direction Recording	15
5.2	Residual-Preserving Bypass Path	15
5.3	Schedule-Driven Register Minimization	15
5.4	Decoupled Scaling-Factor Precision	15
6	Section 6: Team Contributions	16
A	Synthesis Reports for Best ATP Configuration	17
A.1	Timing Report	17
A.2	Area Report	19

Section 1: Wordlength and RMSE Analysis

1.1 Minimum Fractional Wordlength & CORDIC Stage Selection

To set the fixed-point parameters of the EVD bit-true model, we sweep the fractional wordlength and the number of CORDIC micro-rotation stages against all 11 matrices in `Final11Pattern.mat`. A configuration is accepted only when the RMSE of eigenvalues and eigenvectors both remain below 10^{-2} for every test pattern:

$$\text{RMSE}_\lambda = \sqrt{\frac{1}{3} \sum_{i=1}^3 (\hat{\lambda}_i - \lambda_i)^2} \quad \text{RMSE}_v = \sqrt{\frac{1}{9} \sum_{i=1}^3 \|\hat{\mathbf{v}}_i - \mathbf{v}_i\|_2^2}$$

where λ_i and $\hat{\lambda}_i$ are the reference and bit-true eigenvalues, and $\mathbf{v}_i$ and $\hat{\mathbf{v}}_i$ are the associated eigenvectors. We use a two-step search rather than tuning both knobs at once. First, with the CORDIC stage count fixed at 8, we increase the fractional wordlength b until the RMSE criterion is met on all 11 sets; the smallest such value is $b = 11$. Second, with $b = 11$ held fixed, we sweep the CORDIC stage count and find that 8 stages is again the minimum setting that satisfies the same criterion. Thus, the selected bit-true configuration is **11 fractional bits** and **8 CORDIC stages**, which are adopted consistently in the subsequent verification and hardware implementation.

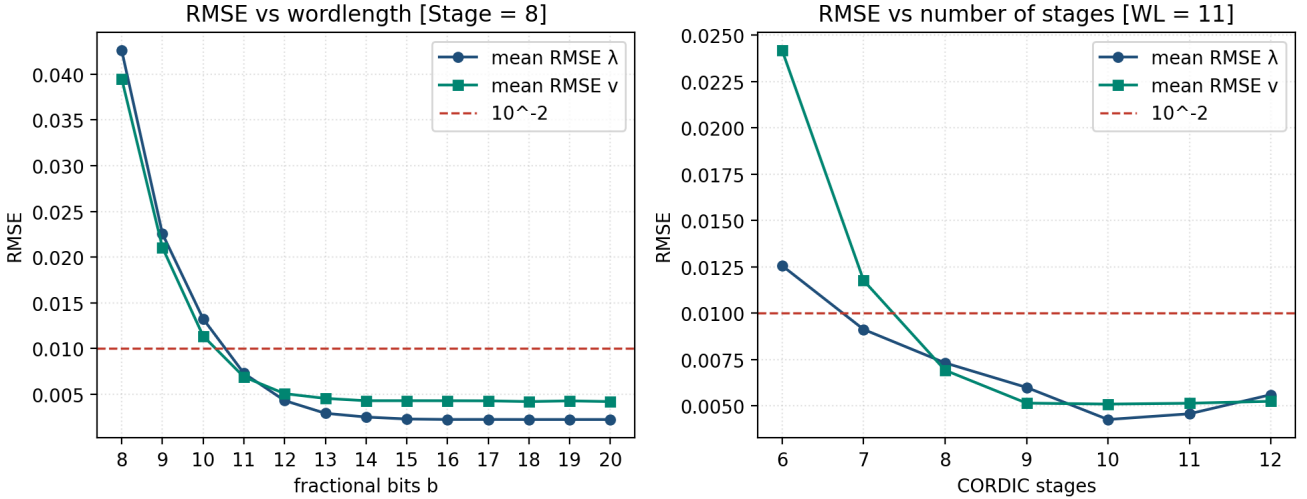


Figure 1.1: RMSE vs. fractional bits ($N_{\text{stage}} = 8$, left) and CORDIC stages ($b = 11$, right).

The two-step sweeps above are useful for intuition, but they do not jointly identify the smallest (b, N_{stage}) pair. Each pass fixes one parameter at the value found earlier, so a configuration that fails in isolation may still satisfy the RMSE criterion when both knobs are set together, and the true minimum combination may be missed.

We therefore perform a two-dimensional sweep over fractional bits b and CORDIC stages N_{stage} . Every grid point is evaluated on all 11 test patterns; a setting passes only when both RMSE_λ and RMSE_v remain below 10^{-2} for every set.

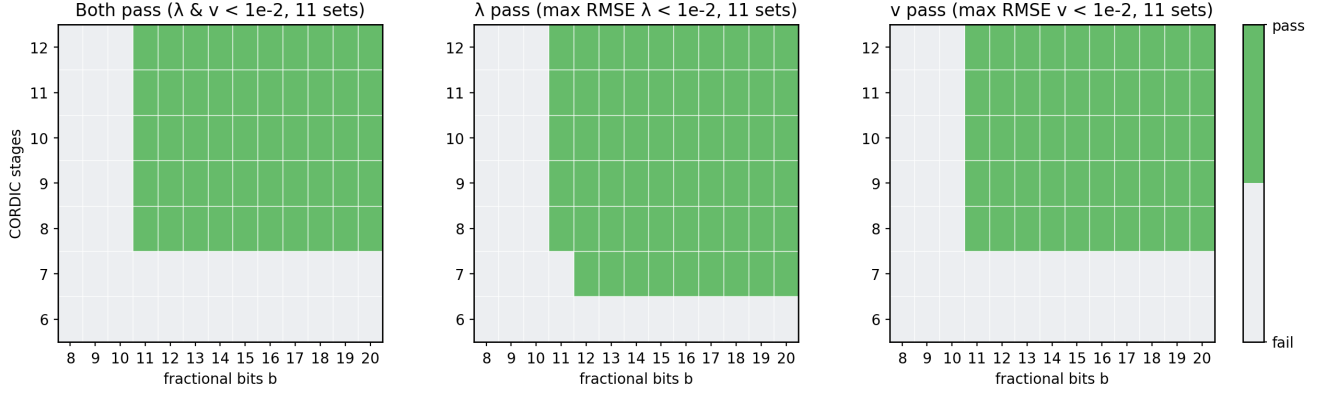


Figure 1.2: 2D pass maps over b and N_{stage} (11 test patterns).

Table 1.1 summarizes the hardware configuration selected from the sweeps above.

Table 1.1: Hardware Configuration

Parameter	Value
B (fractional bits)	11
N_STAGES (CORDIC stages)	8
MAX_ITER (max iterations)	7

1.2 Integer Bit-Width Analysis

With the configuration in Table 1.1, we record the dynamic range of each EVD stage across all 11 patterns and derive the minimum integer wordlength needed at each node. Table 1.2 lists the worst-case integer bit requirement and the corresponding fixed-point format (1 sign bit + integer + B fractional bits).

Table 1.2: Integer Bit-Width Analysis

Node	Min	Max	Max abs	Int. bits	Q format	Total
Quantized input	-2.394	10.239	10.239	4	Q4.11	16
Matrix T/R	-5.117	10.585	10.585	4	Q4.11	16
Eigenvector U	-0.981	1.000	1.000	1	Q1.11	13
CORDIC input	-5.117	10.585	10.585	4	Q4.11	16
CORDIC internal	-10.586	17.432	17.432	5	Q5.11	17
Scaled output	-5.117	10.585	10.585	4	Q4.11	16

1.3 Fixed-Point Bit-True Model

With the fractional wordlength, CORDIC stage count, iteration limit, and integer bit-width requirements established above, we implement a fixed-point bit-true model of the iterative QR EVD datapath. The model applies the formats in Tables 1.1 and 1.2 at each quantization point so that its arithmetic matches the behavior targeted by the RTL design.

This model supports more accurate fixed-point error analysis than a floating-point reference, because quantization, CORDIC rounding, and iterative accumulation are modeled explicitly. It also supplies golden eigenvalues and eigenvectors for RTL verification, so behavior and gate-level simulations can be compared against a consistent fixed-point reference instead of double-precision software alone.

1.4 CORDIC Scaling Factor

Sections 1.1–1.2 do not analyze the impact of quantizing the CORDIC scaling factor. They model gain compensation as $Q_B(X/K)$ only (divide by ideal K), then quantize the quotient. This is intentional, we do not want scaling-factor precision to enlarge global fractional bits B and widen the entire systolic datapath. Therefore, Extra bits are spent only at this multiply-and-round point, not across the systolic array.

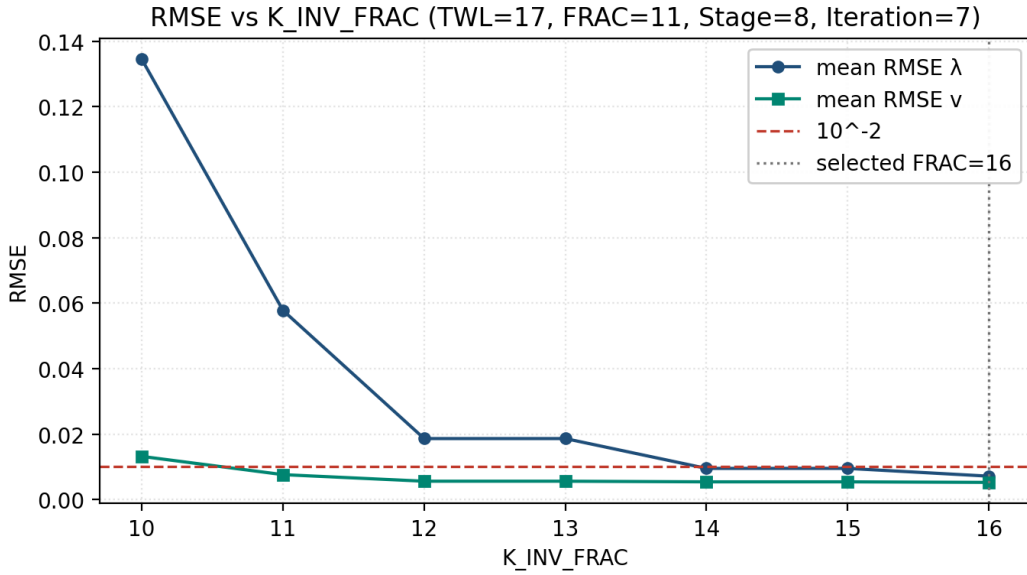


Figure 1.3: RMSE vs. fractional bits of Scaling Factor ($1/K$).

Table 1.3: CORDIC Scaling Factor Configuration

Parameter	Value
DATA_FRAC (datapath fractional bits)	11
DATA_WIDTH (datapath total bits)	17
K_INV_FRAC (fractional bits for $1/K$)	16
K_INV_DW (total bits for $1/K$)	17

1.5 CSD vs Constant Multiplier

For CORDIC gain compensation ($X \times (1/K)$), we compared a hand-designed CSD multiplier with a constant multiplier left for Design Compiler to optimize. In our synthesis experiments, the CSD path introduces truncation error. A custom CSD design that meets the same accuracy requirement is larger than the constant multiplier optimized by Design Compiler. We therefore use the Design Compiler-optimized constant multiplier in the final RTL.

1.6 Bit-True Model Verification

We evaluate the bit-true model on all 11 patterns in `Final11Pattern.mat` with $B = 11$, $N_{\text{stage}} = 8$, $\text{MAX_ITER} = 7$ and $(1/K) = 16$.

Figure 1.4 plots RMSE_λ and RMSE_v per pattern; both remain below 10^{-2} .

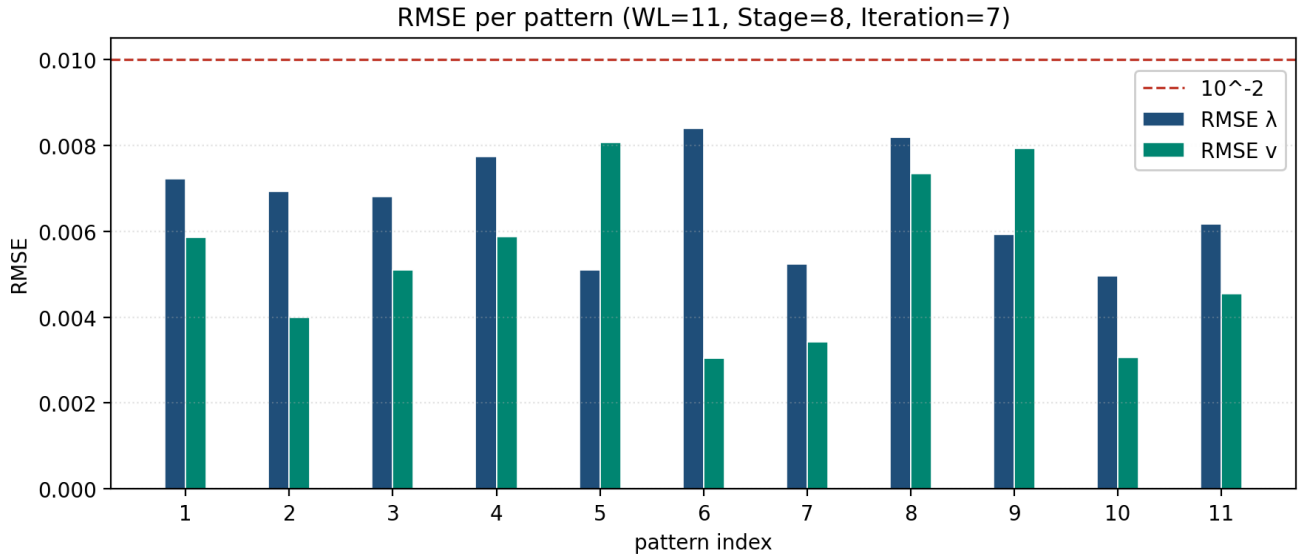


Figure 1.4: RMSE per pattern.

Section 2: Hardware Architecture and Control

2.1 CORDIC PE Architecture

We follow the [Homework 4](#) CORDIC PE design: inputs are preconditioned in the initial stage, and the eight CORDIC micro-rotation stages are unfolded and pipelined to maximize throughput. For this EVD application, **the exact rotation angle θ does not need to be stored**. Vectoring mode records each micro-rotation direction in a rotational direction register (RDR) and rotation mode replays those directions on the remaining matrix entries.

All θ -related arithmetic in the CORDIC datapath can therefore be removed, improving PPA (power, performance, and area). Figure 2.1 shows the resulting PE architecture.

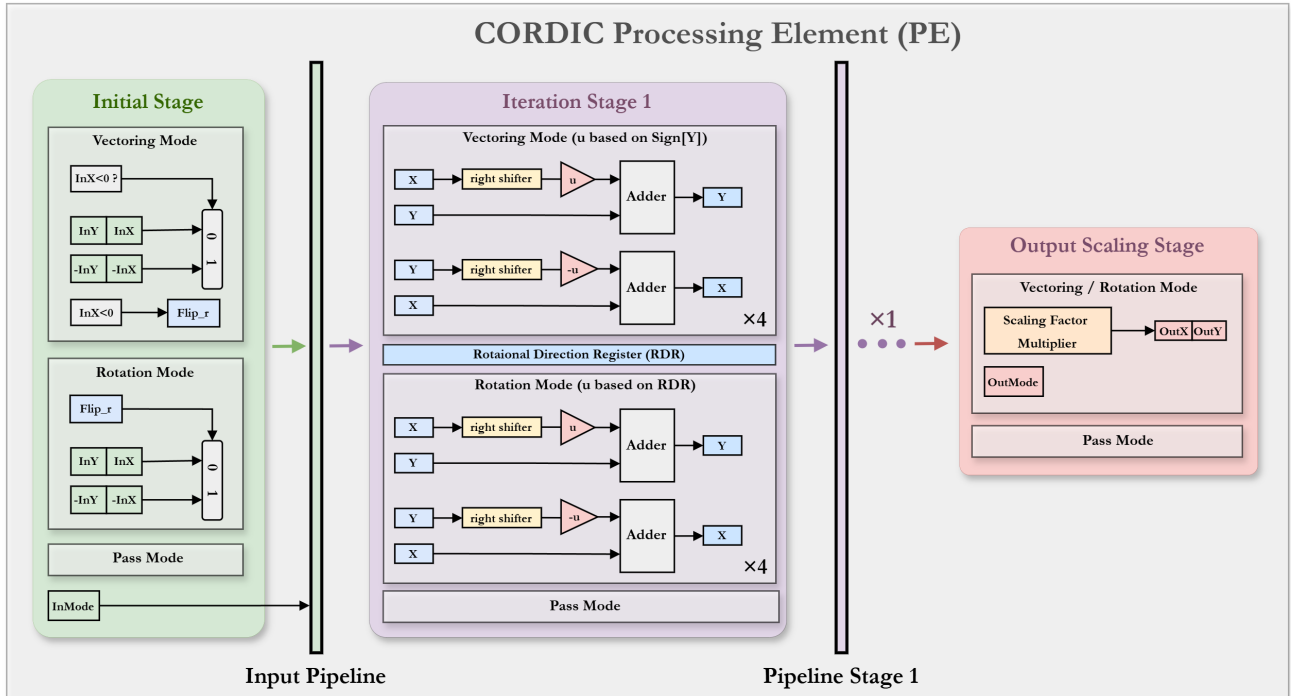


Figure 2.1: CORDIC PE Architecture.

Figure 2.1 shows that the second iteration stage feeds directly into the output scaling stage. For better PPA, the two pipeline stages need not perform the same number of micro-rotations. The first iteration stage can execute five micro-rotations and the second stage three, while still completing all eight stages before gain compensation.

2.2 Systolic Array-Based QRD Architecture

Each CORDIC PE produces one result every two cycles, so the triangular QRD systolic array is retimed around this two-cycle latency. **We remove the delay unit at the bottom-right diagonal position** to keep the array outputs aligned. The detailed timing schedule is given in the next section.

Figure 2.2 shows the resulting array: three CORDIC PE blocks (2-cycle), one single-cycle Delay Unit, and one two-cycle Delay 2 Unit.

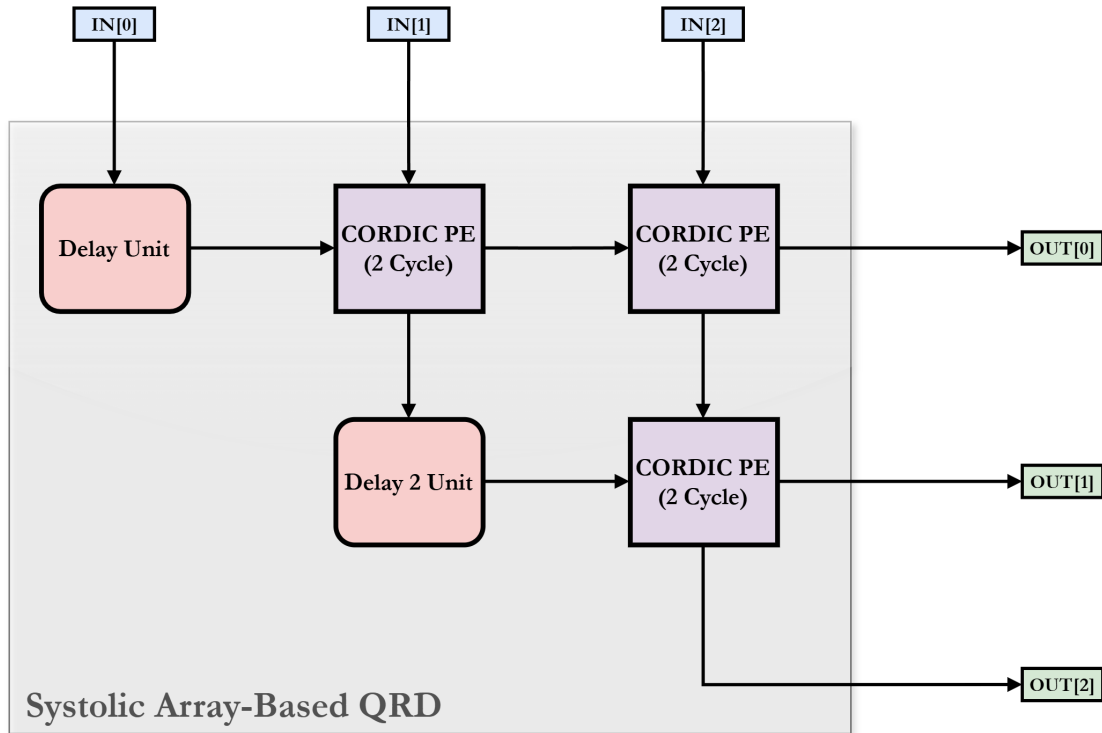


Figure 2.2: Systolic array-based QRD architecture.

2.3 EVD Architecture

The top-level EVD module wraps the QRD systolic array with an input delay unit (IDU), matrix registers, and a three-state FSM. Table 2.1 summarizes the control flow. Figure 2.3 gives the cycle-by-cycle hardware schedule.

Table 2.1: EVD FSM States

State	Function
IDLE	When InValid is asserted, stream the external 3×3 input matrix into the systolic array over three cycles, then enter PROCESS.
PROCESS	Run seven QR iterations. Each iteration takes 15 cycles: feed U for eigenvector update, feed R^T for T update, then feed the updated T back into the array.
OUT	After the seventh iteration, stream the eigenvalues and the three rows of U (eigenvectors) to the output ports over four cycles, then return to IDLE.

In PROCESS, the FSM counter (cnt) schedules reads from the U Register, updates to the Output Buffer, and systolic handshaking so that inputs and outputs remain aligned with the two-cycle CORDIC PE latency. Figure 2.3 tabulates this schedule for the IDLE load phase and one PROCESS iteration.

It is worth noting that lifetime-table analysis of the Output Buffer shows that the minimum number of registers required is **five**.

Cycle	Input Data			Output Data			Output Buffer					U Register								
	InD0	InD1	InD2	OutD0	OutD1	OutD2	0	1	2	3	4	[0][0]	[0][1]	[0][2]	[1][0]	[1][1]	[1][2]	[2][0]	[2][1]	[2][2]
0	T00	T10	T20									I	0	0	0	I	0	0	0	I
1	T01	T11	T21									I	0	0	0	I	0	0	0	I
2	T02	T12	T22									I	0	0	0	I	0	0	0	I
0	U00	U10	U20									I	0	0	0	I	0	0	0	I
1	U01	U11	U21									I	0	0	0	I	0	0	0	I
2	U02	U12	U22	R00								I	0	0	0	I	0	0	0	I
3				R01			R00					I	0	0	0	I	0	0	0	I
4				R02	R10	R20	R00	R01				I	0	0	0	I	0	0	0	I
5	R00	R01	R02	U00	R11	R21	R00	R01	R02	R10	R20	U00	0	0	0	I	0	0	0	I
6	R10	R11	R12	U01	R12	R22	R11	R21	R02	R10	R20	U00	U01	0	0	I	0	0	0	I
7	R20	R21	R22	U02	U10	U20		R21	R22	R10	R20	U00	U01	U02	U10	I	0	U20	0	I
8					U11	U21						U00	U01	U02	U10	U11	0	U20	U21	I
9					U12	U22						U00	U01	U02	U10	U11	U12	U20	U21	U22
10				T00								U00	U01	U02	U10	U11	U12	U20	U21	U22
11				T01			T00					U00	U01	U02	U10	U11	U12	U20	U21	U22
12	T00	T10	T20	T02	T10	T20	T00	T01				U00	U01	U02	U10	U11	U12	U20	U21	U22
13	T01	T11	T21		T11	T21	T00	T01	T02			U00	U01	U02	U10	U11	U12	U20	U21	U22
14	T02	T12	T22		T12	T22	T00	T01	T02	T11		U00	U01	U02	U10	U11	U12	U20	U21	U22

Figure 2.3: Hardware timing schedule (IDLE input load and one PROCESS iteration).

According to the Figure 2.3, One complete EVD run finishes in **112 clock cycles** from input load to the last output word. Table 2.2 breaks this down by FSM phase.

Table 2.2: EVD Latency

Phase	Cycles
IDLE (input load)	3
PROCESS (7 iterations $\times$ 15 cycles)	105
OUT (eigenvalues and eigenvectors)	4
Total	112

A three-state FSM controls the datapath according to the schedule in Figure 2.3. The pre-processor selects the systolic-array inputs on each cycle (external matrix, U Register, or Output Buffer), and the IDU aligns them with the two-cycle CORDIC PE latency. R and T are staged in an Output Buffer. U is stored in a U Register. In PROCESS, the FSM reads both buffers and feeds the array for each QR iteration. In OUT, eigenvalues come from the Output Buffer and eigenvectors from the U Register. Figure 2.4 shows the top-level architecture.

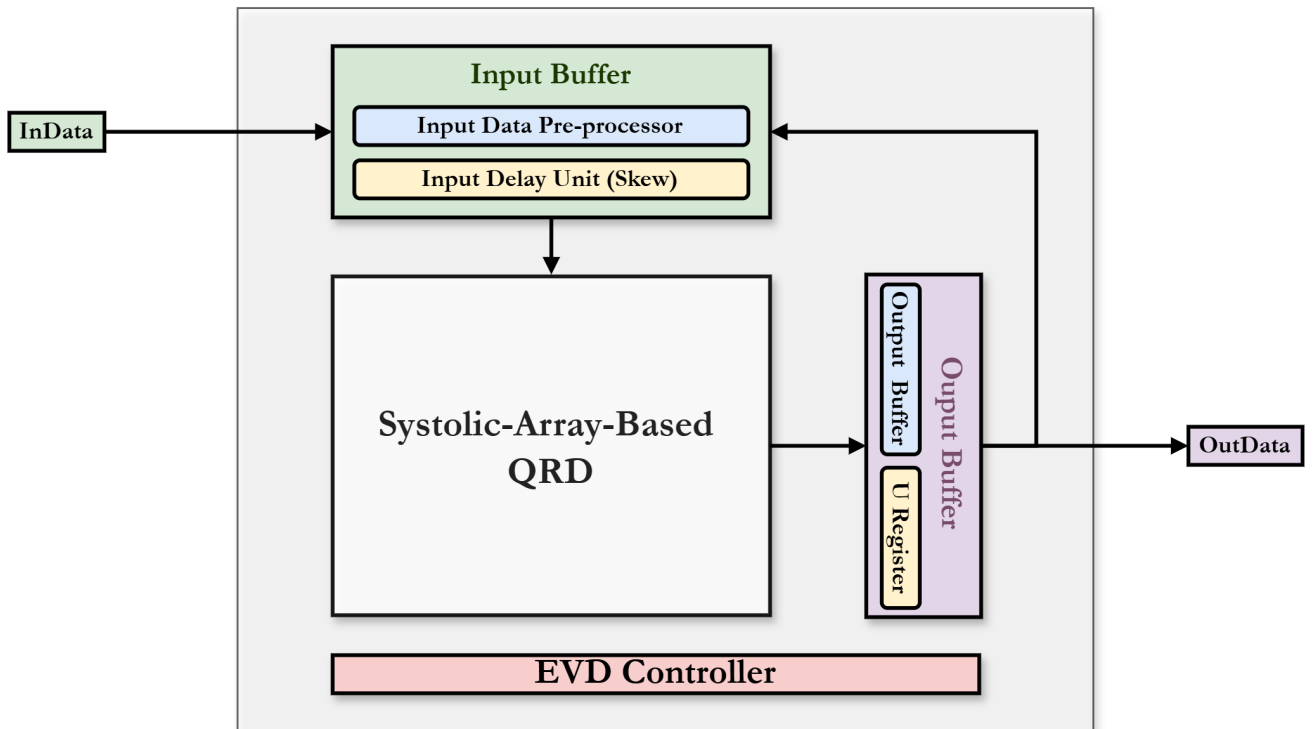


Figure 2.4: Top-level EVD architecture.

2.4 Detailed Transpose Buffer (Output Buffer)

The Output Buffer is the transpose buffer that feeds $\mathbf{R}^T$ back into the array to form $\mathbf{RQ}$. It is simply the direct hardware realization of the timing schedule of Fig. 2.3: the whole iterative operation is sequenced by a single FSM counter cnt , which on each cycle selects both the register written from the array output and the registers read back into the array. No separate addressing logic is needed; cnt alone drives the write and the transposed read-back.

Figure 2.5 shows the buffer at $\text{cnt} = 5, 6$, and 7; the circled cnt value marks when each register was written, and the five registers are read in transposed order to form the three input rows of $\mathbf{R}^T$.

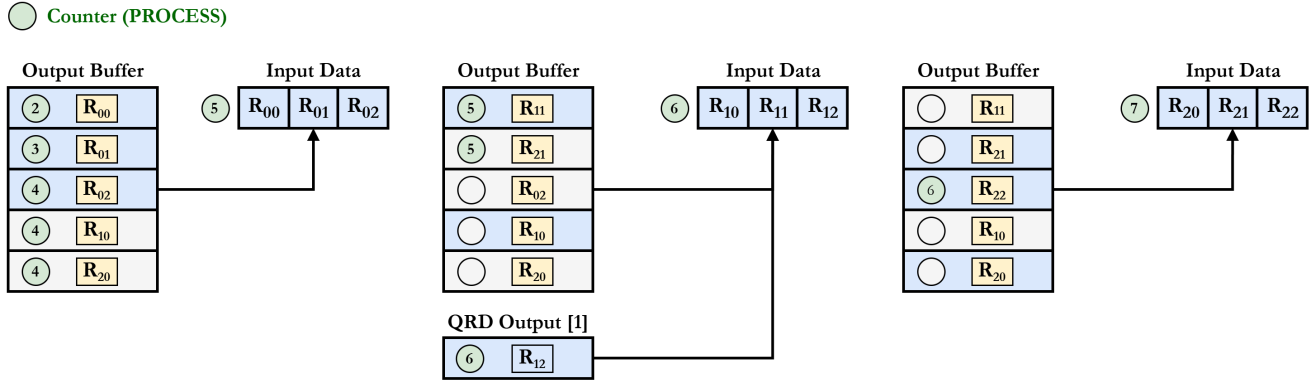


Figure 2.5: Transpose buffer (Output Buffer) Block Diagram.

Section 3: Hardware Simulation

3.1 Behavioral Simulation

We verify the RTL with Synopsys VCS and inspect waveforms in Verdi. Figure 3.1 shows the IDLE phase: InValid is held for three cycles while the input matrix is streamed column-by-column on InData. Figure 3.2 shows the OUT phase: after processing completes, OutValid is asserted for four cycles while the eigenvalues and the three rows of U appear on OutData.



Figure 3.1: RTL simulation waveform: input matrix load (IDLE).

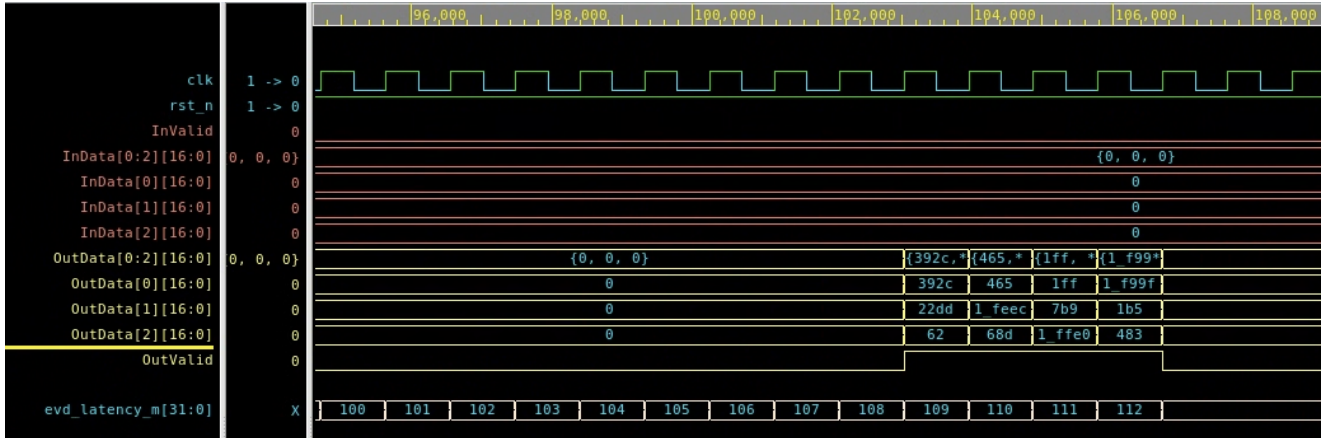


Figure 3.2: RTL simulation waveform: eigenvalue/eigenvector output (OUT).

The testbench exposes a latency monitor `evd_latency_m` that increments on every clock cycle from the rising edge of InValid until the falling edge of OutValid. As shown in the waveforms, one complete EVD run finishes in **112 clock cycles (M)**, consistent with the latency breakdown in Table 2.2.

For the required **test pattern 8**, the behavioral simulation yields $RMSE_\lambda = 5.923380 \times 10^{-3}$ and $RMSE_v = 7.921108 \times 10^{-3}$, both below the 10^{-2} criterion.

3.2 Post-Synthesis Simulation

We run gate-level simulation at the best-ATP from section 6, whose clock period T_s is **923,437 fs** (see the time scale in the upper-right corner of Figures 3.3 and 3.4). The waveforms show that the results are fully consistent with the behavioral simulation.

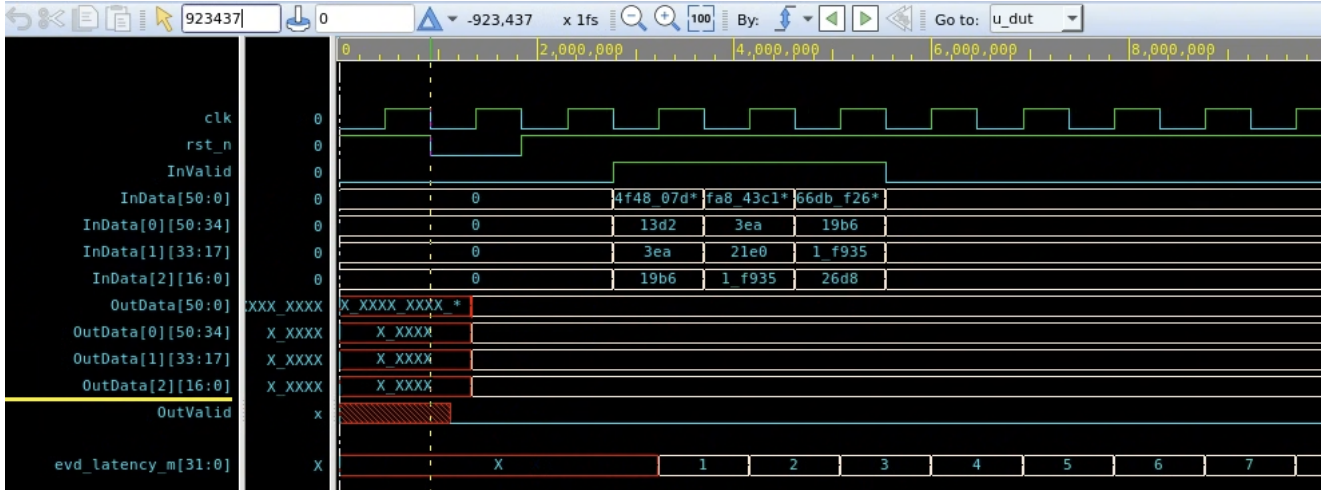


Figure 3.3: Post-synthesis simulation waveform.

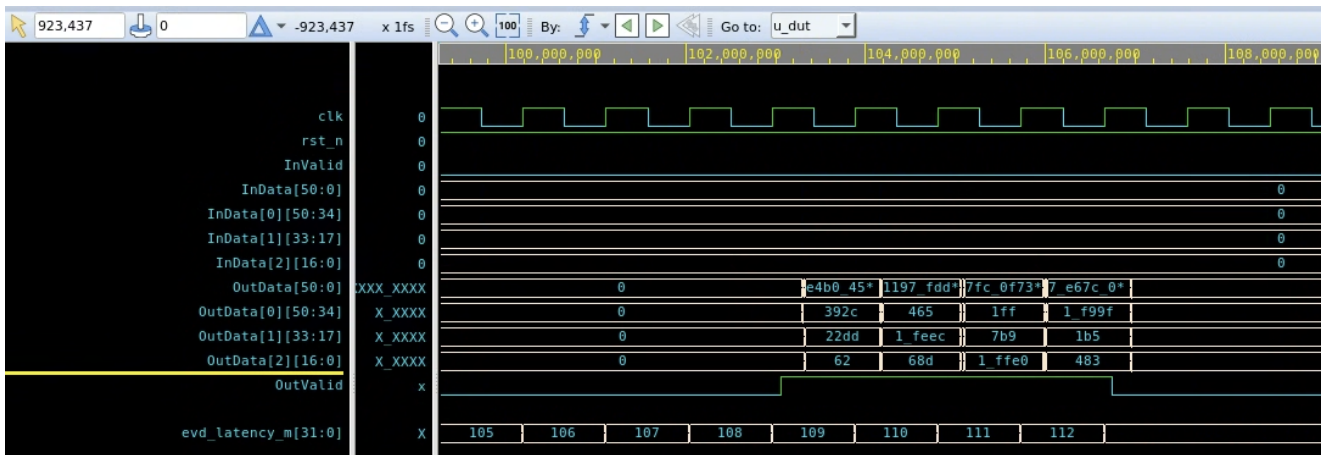


Figure 3.4: Post-synthesis simulation waveform.

3.3 Fixed-Point vs. Hardware Consistency

Hardware outputs match the fixed-point bit-true model exactly on every test pattern.

Figure 1.4 plots the per-pattern RMSE against the reference; both RMSE_A and RMSE_V remain below 10^{-2} for all 11 patterns. Figure 3.5 shows pattern 8 as an example.

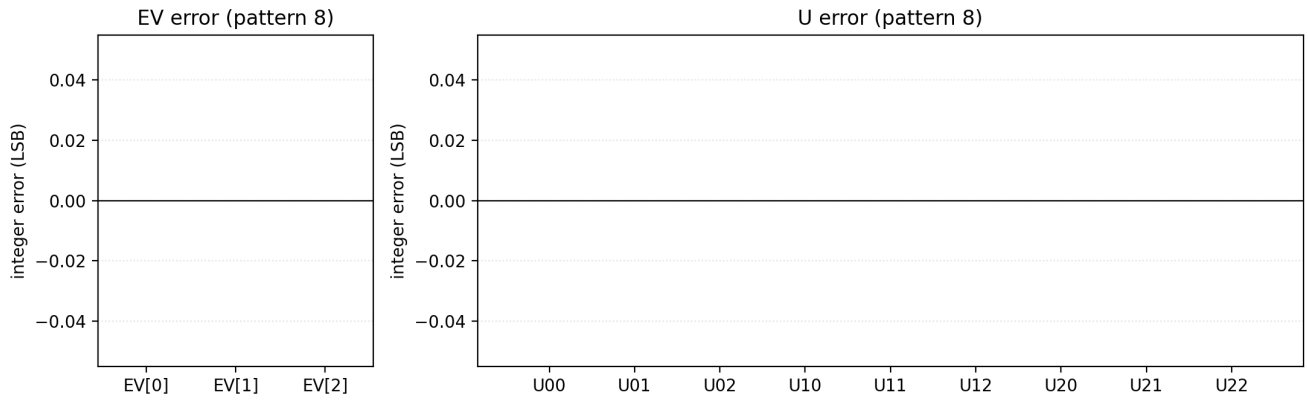


Figure 3.5: RTL vs. bit-true model error (pattern 8).

Section 4: Area-Time Product

The Area-Time Product (ATP) is used to evaluate the trade-off between hardware area and execution time. For each synthesis result, ATP is calculated as

$$\text{ATP} = \text{Area} \times \text{Latency} \times T_{\text{clk}}, \quad (4.1)$$

where the latency is 112 clock cycles and T_{clk} is the clock period in nanoseconds. A lower ATP indicates a better balance between area and timing performance.

Table 4.1: Area-Time Product for Different Synthesis Constraints

Test	Clock Period (ns)	Area	Latency (cycles)	Execution Time (ns)	ATP
1	0.8	6,499.5	112	89.6	582,355.2
2	0.9	5,601.9	112	100.8	564,671.5
3	1.0	4,805.8	112	112	538,249.6
4	1.1	4,759.3	112	123.2	586,345.8
5	1.2	4,646.0	112	134.4	624,422.4
6	1.3	4,564.3	112	145.6	664,562.1
7	0.95	4,962.4	112	106.4	527,999.4
8	0.975	4,892.8	112	109.2	534,293.8
9	1.025	4,846.0	112	114.8	556,320.8
10	0.925	4,995.9	112	103.6	517,575.2
11	0.9225	5,016.6	112	103.32	518,315.1
12	0.9375	4,968.0	112	105	521,640.0
13	0.9625	4,923.7	112	107.8	530,774.9
14	0.9125	5,202.9	112	102.2	531,736.4
15	0.91875	5,133.1	112	102.9	528,196.0
16	0.93125	4,996.5	112	104.3	521,135.0
17	0.921875	5,000.8	112	103.25	516,332.6
19	0.928125	5,018.7	112	103.95	521,693.9
23	0.92225	5,013.6	112	103.292	517,864.8
24	0.92275	5,025.1	112	103.348	519,334.0
25	0.92325	5,013.3	112	103.404	518,395.3
26	0.9235	4,994.0	112	103.432	516,539.4
27	0.92375	5,003.0	112	103.46	517,610.4
28	0.923375	5,009.5	112	103.418	518,072.5
29	0.923437	4,978.0	112	103.425	514,849.4
30	0.923625	5,027.0	112	103.446	520,023.0
31	0.923406	5,004.3	112	103.421	517,552.1
32	0.923438	4,983.6	112	103.425	515,429.1
33	0.923469	4,991.3	112	103.429	516,242.8

Among the evaluated synthesis constraints, the minimum ATP is **514,849.4**, obtained with a clock period of 0.923 437 ns, an area of 4,978.0, and a latency of 112 clock cycles. Therefore, this configuration currently provides the best area–timing trade-off.

Section 5: Design Innovations

5.1 Angle-Free CORDIC via Rotational Direction Recording

A conventional CORDIC carries an explicit angle path: each micro-rotation adds or subtracts a stored $\arctan(2^{-i})$ constant, requiring an arctangent ROM, an angle register, and an angle adder. In the iterative QR EVD the Givens angle is never needed as a number, only its *action* on the matrix. We therefore record only the per-stage rotation directions in an 8-bit *rotational direction register* (RDR) during vectoring, and replay them during rotation, reproducing the same rotation without θ .

5.2 Residual-Preserving Bypass Path

When a PE completes a vectoring operation, its Y output is the eliminated sub-diagonal entry, which is ideally zero but in fixed-point arithmetic is only a small residual. This residual cannot simply be forced to zero, since clamping a value that is not truly zero injects its own error. Yet if the residual is allowed to flow into the next PE and be rotated, the rotation mixes X into Y through successive shift-adds and amplifies the small residual into a much larger error. Neither clamping nor rotating the residual is acceptable.

We resolve this with a bypass path. The downstream PE is preceded by a delay unit that watches the upstream OutMode: a value produced by a vectoring operation is tagged, and after a two-cycle alignment matching the PE latency the tag is raised as a Pass flag. When Pass is asserted, the PE skips every micro-rotation and the gain-scaling multiply, forwarding X and Y unchanged.

The residual is therefore neither clamped nor amplified: it is carried through untouched, keeping the accumulated error bounded across iterations, while the bypassed cycle additionally saves the rotation and scaling switching activity.

5.3 Schedule-Driven Register Minimization

Pipelining each CORDIC PE into two stages raises throughput but skews the array outputs, so extra alignment registers are needed at the inputs and the transpose/output buffer. Their count would dominate the area. We therefore size the buffers from the cycle-by-cycle schedule of Fig. 2.3 rather than by worst-case guessing. A lifetime analysis over that schedule shows the transpose/output buffer needs only **five** registers. The same schedule also exposes a redundant delay unit at the bottom-right diagonal, whose value is already aligned by the adjacent two-cycle PE latency; it is replaced by a direct wire (Fig. 2.2).

5.4 Decoupled Scaling-Factor Precision

As detailed in Section 1.4, CORDIC gain compensation is the only point that demands extra fractional precision. Rather than widening the global fractional width B and inflating every register along the systolic datapath, we keep the datapath at B and spend the additional bits solely at the final $X \times (1/K)$ multiply-and-round, so the precision cost is confined to a single node instead of being paid array-wide.

Section 6: Team Contributions

Table 6.1: Team Contributions and Work Distribution

Work Item	MING-HONG,LIN	YING-CHENG,CHEN
Algorithm Research and Configuration Analysis	✓	✓
Fixed-Point Bit-True Model Development	✓	✓
Total Word-Length and Accuracy Analysis	✓	✓
Hardware Architecture and Control Flow Design	✓	✓
RTL Design	✓	✓
Testbench Development and Unit Verification	✓	✓
Behavioral Simulation	✓	✓
Logic Synthesis and Area/Timing Analysis	✓	✓
Post-Synthesis Simulation	✓	✓
Hardware Optimization	✓	✓
Area-Time Product Evaluation	✓	✓
Result Analysis and Visualization	✓	✓
Report Writing	✓	✓
Weight	50%	50%

Synthesis Reports for Best ATP Configuration

A.1 Timing Report

Information: Updating design information... (UID-85)		

Report : timing		
-path full		
-delay max		
-max_paths 1		
Design : EVD		
Version: W-2024.09-SP2		
Date : Tue Jun 16 03:16:10 2026		

Operating Conditions: ff0p88v125c Library: N16ADFP_StdCellff0p88v125c_ccs		
Wire Load Model Mode: segmented		
Startpoint: u_QRD/ROW0_COL2/Y_r_reg[1][10]		
(rising edge-triggered flip-flop clocked by clk)		
Endpoint: TEMP_reg[2][16]		
(rising edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: max		
Des/Clust/Port	Wire Load Model	Library

CORDIC_PE_2	ZeroWireload	N16ADFP_StdCellff0p88v125c_ccs
EVD	ZeroWireload	N16ADFP_StdCellff0p88v125c_ccs
CORDIC_PE_0	ZeroWireload	N16ADFP_StdCellff0p88v125c_ccs

Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.50	0.50
u_QRD/ROW0_COL2/Y_r_reg[1][10]/CP (DFCNQD1BWP20P90)	0.00	0.50 r
u_QRD/ROW0_COL2/Y_r_reg[1][10]/Q (DFCNQD1BWP20P90)	0.05	0.55 r
U3142/ZN (INVD1BWP20P90)	0.01	0.56 f
U3031/Z (OR2D1BWP20P90)	0.02	0.57 f
U3161/ZN (AOI21D1BWP20P90)	0.01	0.59 r
U3162/ZN (OAI21D1BWP16P90)	0.01	0.60 f
U3163/ZN (AOI21D1BWP16P90)	0.02	0.61 r
U3360/ZN (CKND2BWP16P90)	0.01	0.63 f
U1957/ZN (XNR2D1BWP20P90)	0.02	0.65 r
U2351/Z (MUX2D2BWP20P90)	0.02	0.67 r
U1808/Z (XOR2D1BWP20P90)	0.02	0.69 f
U1867/ZN (NR2D1BWP20P90)	0.01	0.70 r
U4350/ZN (OAI21D1BWP16P90)	0.01	0.71 f
U4351/ZN (AOI21D1BWP16P90)	0.01	0.72 r
U4356/ZN (OAI21D1BWP16P90)	0.01	0.73 f
U1698/ZN (AOI21D1BWP16P90)	0.01	0.75 r
U1693/ZN (OAI21D1BWP20P90LVT)	0.01	0.76 f
U4555/ZN (AOI21D2BWP16P90)	0.01	0.77 r
U1261/ZN (OAI21D2BWP16P90)	0.01	0.78 f
U4568/C0 (FA1D1BWP16P90)	0.02	0.80 f
U4572/C0 (FA1D1BWP16P90)	0.02	0.82 f
U4581/S (FA1D1BWP16P90)	0.03	0.86 r

Figure A.1: Critical-path timing report (max delay), part 1.

U4568/CO (FA1D1BWP16P90)	0.02	0.80 f
U4572/CO (FA1D1BWP16P90)	0.02	0.82 f
U4581/S (FA1D1BWP16P90)	0.03	0.86 r
U4573/Z (MUX2D1BWP16P90)	0.02	0.88 r
U1421/Z (XOR2D1BWP16P90)	0.02	0.89 f
U2535/Z (OR2D1BWP16P90)	0.01	0.91 f
U1409/ZN (AOI21D1BWP16P90)	0.01	0.92 r
U1607/ZN (OAI21D1BWP16P90LVT)	0.01	0.93 f
U1597/ZN (AOI21D1BWP20P90LVT)	0.01	0.94 r
U1590/ZN (OAI21D1BWP16P90LVT)	0.01	0.95 f
U1155/ZN (AOI21D1BWP16P90LVT)	0.01	0.96 r
U2428/ZN (OAI21D2BWP16P90LVT)	0.01	0.97 f
U4796/ZN (AOI21D1BWP16P90)	0.01	0.99 r
U4797/Z (XOR2D4BWP16P90)	0.03	1.01 f
U4842/CO (FA1D1BWP16P90)	0.02	1.03 f
U4835/S (FA1D1BWP16P90)	0.04	1.07 r
U4845/S (FA1D1BWP16P90)	0.03	1.10 f
U2338/Z (OR2D1BWP16P90)	0.02	1.12 f
U1368/ZN (AOI21D1BWP16P90)	0.01	1.13 r
U1365/ZN (OAI21D1BWP16P90)	0.01	1.14 f
U1541/ZN (AOI21D1BWP20P90LVT)	0.01	1.15 r
U1537/ZN (OAI21D1BWP16P90LVT)	0.01	1.16 f
U3095/ZN (AOI21D1BWP16P90LVT)	0.01	1.17 r
U2273/ZN (OAI21D2BWP16P90LVT)	0.01	1.18 f
U1357/ZN (AOI21D2BWP20P90LVT)	0.01	1.19 r
U1530/ZN (OAI21D2BWP16P90LVT)	0.01	1.20 f
U1347/ZN (AOI21D2BWP20P90LVT)	0.01	1.21 r
U1524/ZN (OAI21D2BWP16P90LVT)	0.01	1.21 f
U1311/ZN (AOI21D1BWP16P90LVT)	0.01	1.23 r
U1309/ZN (OAI21D1BWP16P90LVT)	0.01	1.24 f
U1999/CO (FA1D1BWP16P90LVT)	0.02	1.26 f
U1169/CO (FA1D1BWP16P90LVT)	0.02	1.28 f
U5871/ZN (INVD1BWP16P90)	0.01	1.29 r
U1899/ZN (AOI22D1BWP20P90)	0.01	1.30 f
U5806/ZN (OAI21D1BWP20P90)	0.01	1.31 r
TEMP_reg[2][16]/D (DFCNQD1BWP20P90)	0.00	1.31 r
data arrival time		1.31
clock clk (rise edge)	0.92	0.92
clock network delay (ideal)	0.50	1.42
clock uncertainty	-0.10	1.32
TEMP_reg[2][16]/CP (DFCNQD1BWP20P90)	0.00	1.32 r
library setup time	-0.02	1.31
data required time		1.31
data required time		1.31
data arrival time		-1.31
slack (MET)		0.00

1

Figure A.2: Critical-path timing report (max delay), part 2. Slack is 0.00 ns (MET).

A.2 Area Report

```
*****
Report : area
Design : EVD
Version: W-2024.09-SP2
Date   : Tue Jun 16 03:16:10 2026
*****

Library(s) Used:

  N16ADFP_StdCellff0p88v125c_ccs (File: /usr/cad/ADFP/Executable_Package/Collaterals/IP/stdcell/N16ADFP_StdCell/CCS/N16ADFP_StdCellff0p88v125c_ccs.db)

Number of ports:          252
Number of nets:          14665
Number of cells:          13792
Number of combinational cells: 13181
Number of sequential cells:   609
Number of macros/black boxes:    0
Number of buf/inv:          2321
Number of references:        152

Combinational area:        4315.161688
Buf/Inv area:              380.246416
Noncombinational area:     662.826261
Macro/Black Box area:      0.000000
Net Interconnect area:     undefined (Wire load has zero net area)

Total cell area:           4977.987949
Total area:                undefined
1
```

Figure A.3: Post-synthesis area report.